

## TRABAJO DE CAMPO 01 AL 04

### Parte 01: Acta de Reunión

Para iniciar con el Trabajo de Campo de la Semana se le solicita llenar la siguiente acta de reunión:

| ACTA DE REUNIÓN                                                                                                                 |                                             |                          |
|---------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------|--------------------------|
| <b>Fecha:</b><br>06/09/2026                                                                                                     | <b>Hora:</b><br>10:00 am                    | <b>Lugar:</b><br>Virtual |
| <b>ASISTENTES</b>                                                                                                               |                                             |                          |
| <b>NOMBRES Y APELLIDOS</b>                                                                                                      | <b>CÓDIGO UPN</b>                           | <b>ASISTIÓ</b>           |
| 1. MANUEL ALONSO APOLAYA ESPINOZA                                                                                               | N00269905                                   | SI                       |
| 2. CONNI DIANA JIMENEZ REQUES                                                                                                   | N00422384                                   | SI                       |
| 3. WALTER BRYAM RUIZ SALAZAR                                                                                                    | N00371631                                   | SI                       |
| 4. OSMAR ANTONIO VELEZMORO COTILLO                                                                                              | N00369775                                   | SI                       |
| <b>TEMAS</b>                                                                                                                    |                                             |                          |
| 1. Plataforma Distribuida de Sincronización de Inventarios y Logística para Calzado Deportivo                                   |                                             |                          |
| 2.                                                                                                                              |                                             |                          |
| 3.                                                                                                                              |                                             |                          |
| 4.                                                                                                                              |                                             |                          |
| <b>DESARROLLO</b>                                                                                                               |                                             |                          |
| <ul style="list-style-type: none"><li>Se adjunta desarrollo del trabajo de campo, a partir de la hoja 02.</li></ul>             |                                             |                          |
| <b>FIRMA</b>                                                                                                                    |                                             |                          |
| El equipo de trabajo <b>colocando el nombre del líder del equipo</b> , afirma que la información de este documento es verídica. | <b>Grupo:</b> __5__<br><b>Nombre:</b> _____ |                          |

## Plataforma Distribuida de Sincronización de Inventarios y Logística para Calzado Deportivo

Repositorio: <https://github.com/ovelezmoro/shopchoin>

# CAPITULO 01

## Descripción del problema

La falta de sincronización en tiempo real entre el catálogo web y el inventario físico de las sucursales genera diferencias en la disponibilidad de los productos. Esto puede ocasionar sobreventas, quiebres de stock durante la preparación de pedidos y demoras en la atención de clientes que eligen retiro presencial.

## Contexto de la organización o caso seleccionado

El caso corresponde a un operador retail especializado en la venta de calzado deportivo, con sucursales físicas ubicadas en San Miguel, San Isidro, Breña y Bellavista. Todas las tiendas son abastecidas desde un almacén central, por lo que la correcta actualización del inventario es necesaria para coordinar las ventas, reposiciones y pedidos realizados desde la plataforma web.

## Usuarios involucrados

Los principales usuarios de la plataforma serán:

- **Clientes:** consultan productos, disponibilidad y realizan pedidos.
- **Personal de tienda:** verifica stock y gestiona pedidos para retiro presencial.
- **Personal de almacén:** controla el inventario central y las reposiciones.
- **Administrador:** supervisa productos, sucursales, inventarios y movimientos de stock.

## Necesidad identificada

Se necesita contar con una plataforma que permita mantener actualizado el inventario de cada sucursal y del almacén central, de modo que la disponibilidad mostrada al cliente sea confiable y se reduzcan los errores durante la venta, preparación y retiro de pedidos.

## Objetivo de la solución

Desarrollar una aplicación web distribuida que permita sincronizar el inventario entre el almacén central, las sucursales y el catálogo web, mejorando la disponibilidad de la información y reduciendo los problemas de sobreventa, quiebres de stock y demoras en los pedidos.

---

## **Alcance preliminar**

La solución permitirá gestionar productos, consultar el stock disponible por sucursal, actualizar inventarios, registrar movimientos de stock y visualizar la disponibilidad desde el catálogo web. También contemplará la gestión de pedidos con retiro presencial y la sincronización de información entre las sucursales y el almacén central.

## **Limitaciones iniciales**

La primera versión de la plataforma estará enfocada únicamente en la gestión y sincronización de inventarios, productos y pedidos con retiro presencial. No se contemplarán inicialmente funciones avanzadas como integración con proveedores externos, optimización automática de rutas logísticas o análisis predictivo de demanda.

## **Justificación de la pertinencia de desarrollar una aplicación web distribuida**

El desarrollo de una aplicación web distribuida es pertinente porque permite conectar el almacén central, las sucursales y el catálogo web mediante servicios que comparten información de inventario de forma rápida y consistente. Esto facilita la actualización del stock, mejora la disponibilidad de la información y permite que la solución pueda crecer conforme aumenten las sucursales, productos o usuarios.

---

## ***CAPITULO 02***

### **Mapa de navegación**

La aplicación tendrá una navegación organizada según el tipo de usuario y las funciones disponibles:

- **Inicio de sesión**
- **Inicio**
- **Catálogo de productos**
  - Detalle del producto
  - Stock por sucursal
- **Pedidos**
  - Registro de pedido
  - Seguimiento del pedido
  - Retiro en tienda
- **Inventario**
  - Stock por sucursal
  - Movimientos de inventario
  - Reposición de productos
- **Administración**
  - Gestión de productos
  - Gestión de sucursales
  - Gestión de usuarios

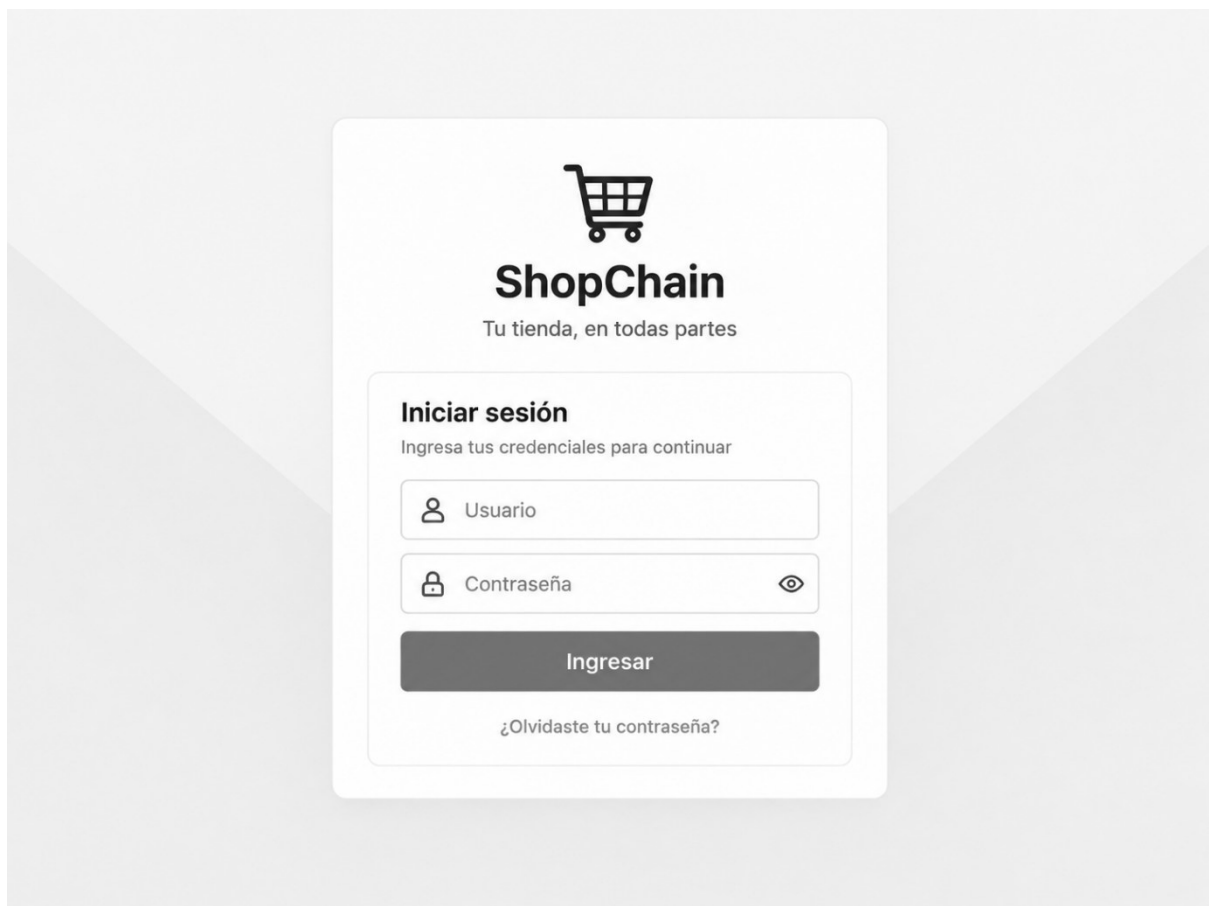
## Estructura de componentes frontend

El frontend será desarrollado con Angular y organizado de manera modular, separando los componentes según las funcionalidades principales del sistema. Esta estructura facilitará el mantenimiento, reutilización y crecimiento de la aplicación.

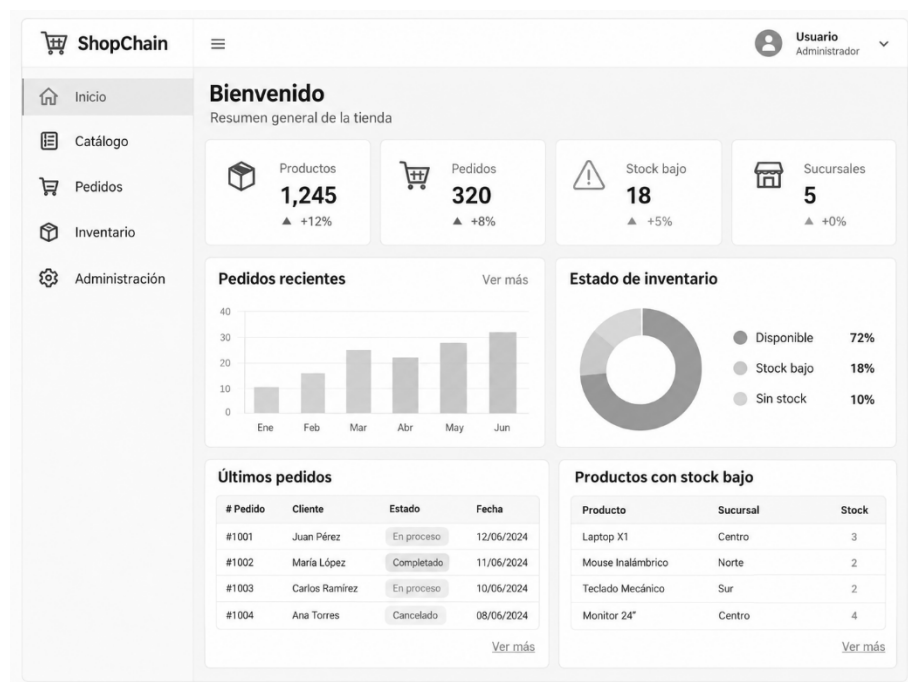
La estructura principal se divide en:

- **auth:** inicio de sesión.
- **home:** pantalla principal de la aplicación.
- **catalog:** catálogo, detalle de productos y disponibilidad por sucursal.
- **orders:** registro, seguimiento y retiro de pedidos.
- **inventory:** consulta de stock, movimientos y reposiciones.
- **administration:** gestión de productos, sucursales y usuarios.
- **core y shared:** contienen elementos técnicos y reutilizables que apoyan a las demás funcionalidades, pero no representan opciones del mapa de navegación.

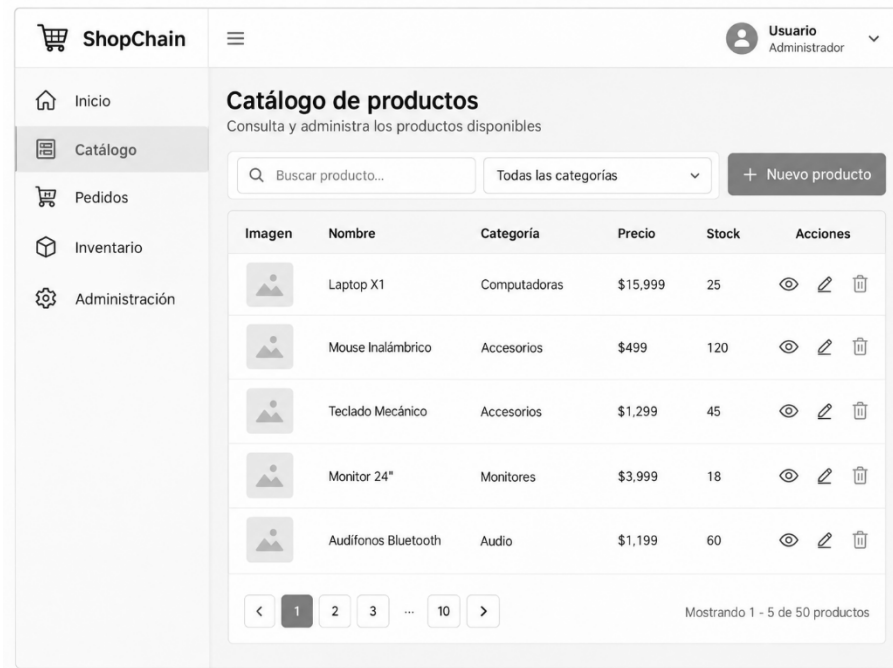
De esta manera, cada sección navegable del sistema tiene su correspondiente conjunto de componentes en Angular.



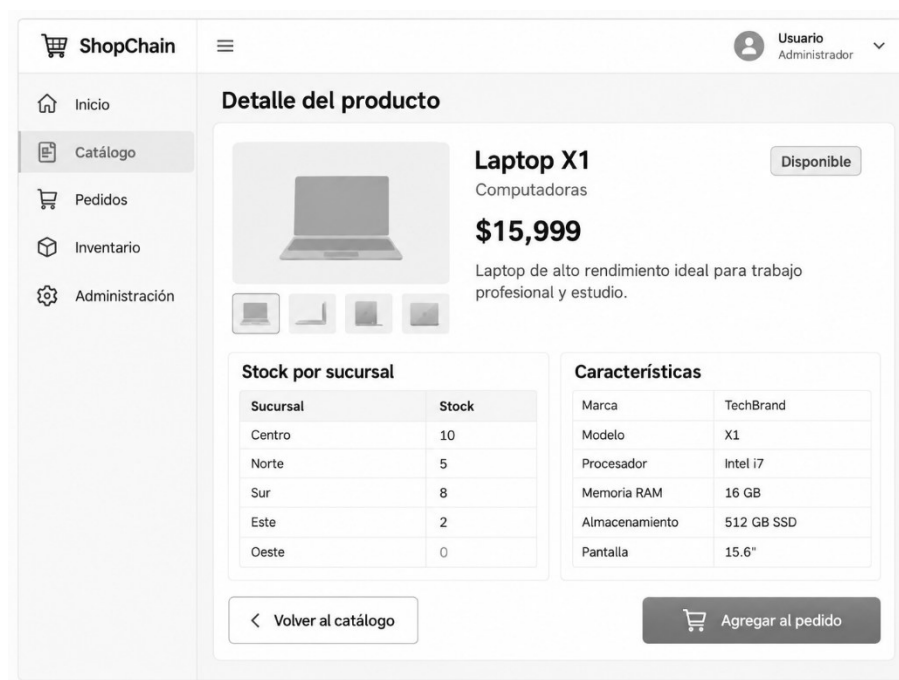
**Login:** formulario de acceso mediante usuario y contraseña.



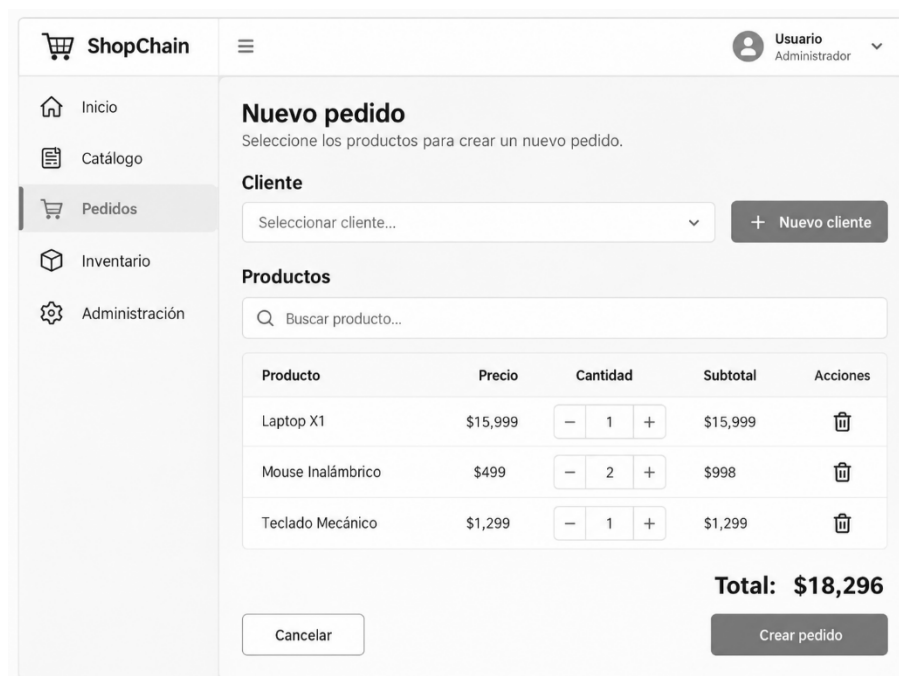
**Inicio:** resumen general de productos, pedidos y estado del inventario.



**Catálogo:** listado de productos con búsqueda, precio y disponibilidad.



**Detalle del producto:** información del producto y stock disponible por sucursal.



**Pedidos:** registro de pedidos.

ShopChain

Inicio

Catálogo

Pedidos

Inventario

Administración



Usuario

Administrador



## Seguimiento de pedido

Consulta el estado de tus pedidos realizados.



Buscar por número de pedido...

| # Pedido | Fecha      | Cliente        | Estado     | Total    | Acciones                                                                            |
|----------|------------|----------------|------------|----------|-------------------------------------------------------------------------------------|
| #1001    | 12/06/2024 | Juan Pérez     | En proceso | \$15,999 |  |
| #1002    | 11/06/2024 | María López    | Completado | \$8,450  |  |
| #1003    | 10/06/2024 | Carlos Ramírez | En proceso | \$23,990 |  |
| #1004    | 09/06/2024 | Ana Torres     | Cancelado  | \$1,999  |  |
| #1005    | 08/06/2024 | Luis García    | Completado | \$5,699  |  |



1

2

3

...

10



Mostrando 1 - 5 de 50 pedidos

**Pedidos:** seguimiento de pedidos realizados.

Inicio

Catálogo

Pedidos

Inventario

Administración

Usuario

Administrador

Inventario

Consulta el stock por producto y sucursal

Buscar producto...

Todas las sucursales

Filtrar

| Producto                 | Categoría | Sucursal   | Stock disponible | Stock mínimo | Estado       | Acciones |
|--------------------------|-----------|------------|------------------|--------------|--------------|----------|
| Zapatilla Running Pro    | Running   | San Miguel | 25               | 10           | ● Disponible |          |
| Zapatilla Urban Flex     | Lifestyle | San Isidro | 6                | 10           | ● Stock bajo |          |
| Zapatilla Court Max      | Tenis     | Breña      | 0                | 5            | ● Sin stock  |          |
| Zapatilla Trail Explorer | Outdoor   | Bellavista | 18               | 8            | ● Disponible |          |
| Zapatilla Air Runner     | Running   | San Miguel | 4                | 10           | ● Stock bajo |          |
| Zapatilla Victory        | Lifestyle | San Isidro | 32               | 10           | ● Disponible |          |
| Zapatilla Mountain Lite  | Outdoor   | Breña      | 3                | 8            | ● Stock bajo |          |
| Zapatilla Classic        | Lifestyle | Bellavista | 0                | 5            | ● Sin stock  |          |

<

1

2

3

4

5

>

Mostrando 1 - 8 de 8 productos

**Inventario:** consulta del stock por producto y sucursal.

ShopChain

Usuario

Administrador

Inicio

Catálogo

Pedidos

Inventario

Administración

Movimientos de stock

Registro de entradas, salidas y reposiciones

Tipo de movimiento

Todos los movimientos

Rango de fechas

01/06/2024 - 30/06/2024

Sucursal

Todas las sucursales

+ Registrar movimiento

| Fecha      | Producto                      | Tipo de movimiento | Cantidad | Sucursal   | Responsable    | Observación                         | Acciones |  |
|------------|-------------------------------|--------------------|----------|------------|----------------|-------------------------------------|----------|--|
| 12/06/2024 | Nike Air Max 270<br>Talla 42  | Entrada            | + 50     | San Miguel | Juan Pérez     | Compra a proveedor                  |          |  |
| 11/06/2024 | Adidas Ultraboost<br>Talla 41 | Salida             | - 5      | San Isidro | Maria López    | Venta tienda física                 |          |  |
| 10/06/2024 | Puma RS-X<br>Talla 40         | Reposición         | + 20     | Breña      | Carlos Ramírez | Reposición por baja de stock        |          |  |
| 09/06/2024 | New Balance 574<br>Talla 42   | Salida             | - 3      | Bellavista | Ana Torres     | Venta online                        |          |  |
| 08/06/2024 | Nike Revolution 6<br>Talla 41 | Entrada            | + 40     | San Isidro | Luis García    | Transferencia desde almacén central |          |  |
| 07/06/2024 | Adidas Superstar<br>Talla 39  | Reposición         | + 15     | San Miguel | Sofía Martínez | Reposición por aumento de demanda   |          |  |
| 06/06/2024 | Puma Caven 2.0<br>Talla 42    | Salida             | - 8      | Breña      | Diego Chávez   | Venta tienda física                 |          |  |
| 05/06/2024 | New Balance 327<br>Talla 40   | Entrada            | + 30     | Bellavista | Valeria Rojas  | Compra a proveedor                  |          |  |
| 04/06/2024 | Nike Court Vision<br>Talla 41 | Salida             | - 6      | San Miguel | Juan Pérez     | Venta online                        |          |  |
| 03/06/2024 | Adidas Runfalcon<br>Talla 40  | Reposición         | + 25     | San Isidro | Maria López    | Ajuste de inventario                |          |  |

<

1

2

3

4

5

...

10

>

Mostrando 1 - 10 de 50 movimientos

**Movimientos de stock:** registro de entradas, salidas y reposiciones.

ShopChain

</

**Administración:** gestión de productos, sucursales y usuarios.

## Crterios bsicos de UI/UX

La interfaz de la aplicacin seguir crterios simples que permitan una navegacin clara y rpida:

- **Diseo consistente:** uso de los mismos colores, botones, tipografas e iconos en todas las pantallas.
- **Navegacin sencilla:** men lateral con acceso directo a los mdulos principales.
- **Jerarqua visual:** ttulos, tablas y acciones principales claramente diferenciadas.
- **Estados visibles:** uso de etiquetas para mostrar estados como disponible, stock bajo, sin stock, en proceso o completado.
- **Diseo responsive:** adaptacin de la interfaz a diferentes tamaos de pantalla.
- **Formularios simples:** campos claramente identificados, validaciones visibles y botones de accin fciles de reconocer.

## Descripcin de formularios principales

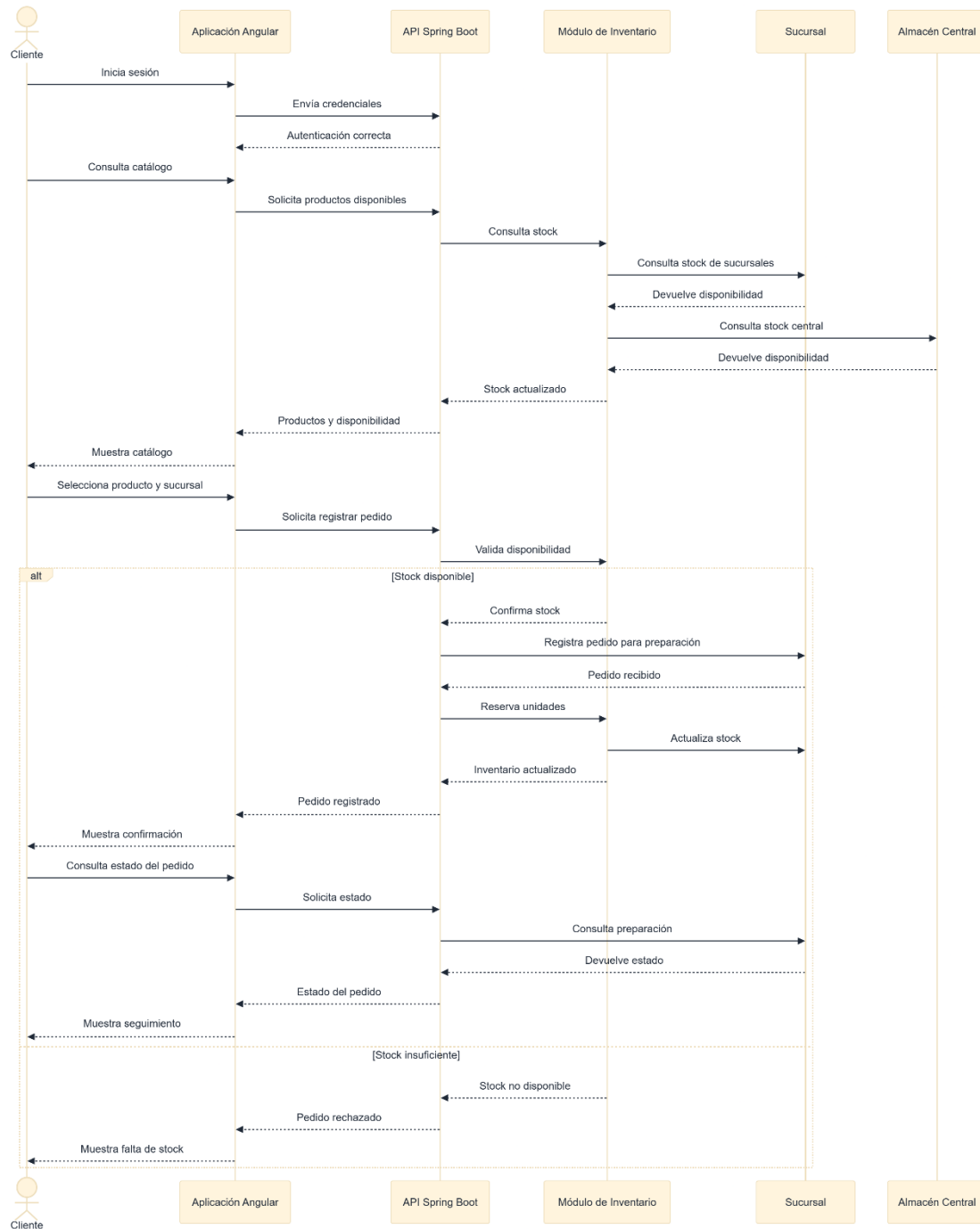
Los formularios principales de la aplicacin sern:

- **Inicio de sesin:** usuario y contrasea.
- **Registro de producto:** nombre, categora, precio, descripcin y stock inicial.
- **Registro de pedido:** cliente, productos, cantidades y sucursal de retiro.
- **Movimiento de stock:** producto, tipo de movimiento, cantidad, sucursal y observacin.
- **Registro de sucursal:** nombre, direccin, ubicacin y estado.
- **Registro de usuario:** nombres, correo, contrasea, rol y estado.

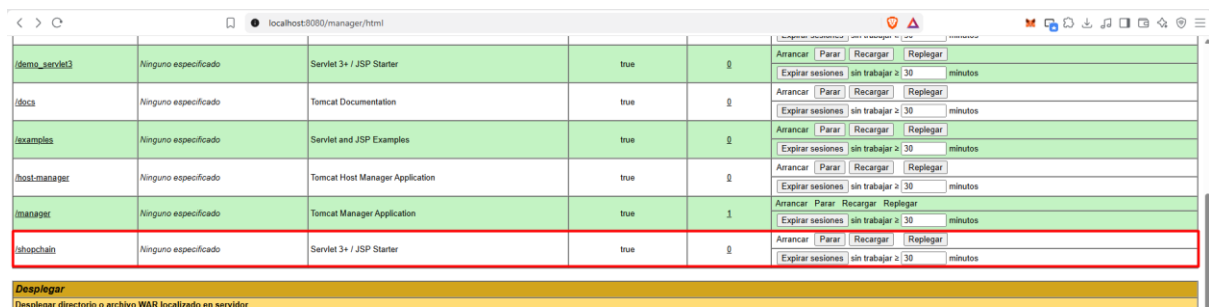
Todos los formularios incluirn validaciones bsicas para evitar registros incompletos o incorrectos.

## Flujo de interacción del usuario

El flujo de interacción del usuario puede representarse mediante un diagrama de secuencia, donde se observa el proceso desde la consulta de productos hasta la actualización del inventario y el seguimiento del pedido.



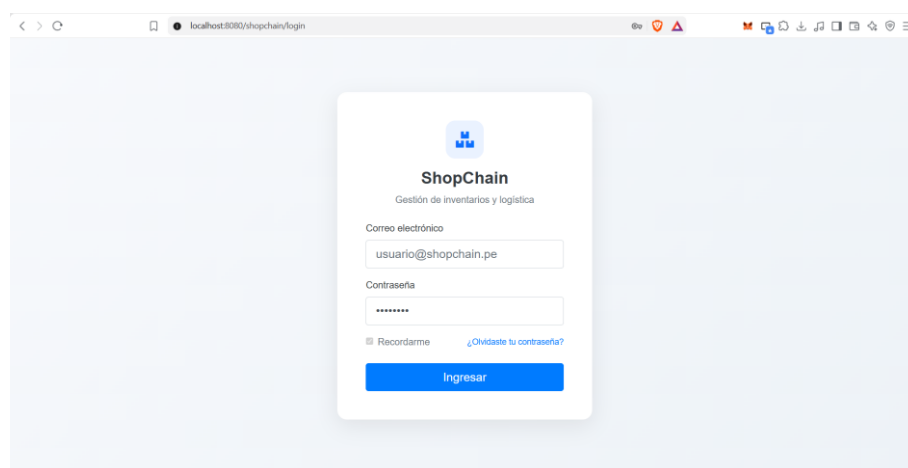
## Capturas del prototipo inicial desarrollado



|                |                      |                                 |      |   |                                                                                        |
|----------------|----------------------|---------------------------------|------|---|----------------------------------------------------------------------------------------|
| /demo_servlet3 | Ninguno especificado | Servlet 3+ / JSP Starter        | true | 0 | Arrancar   Parar   Recargar   Replegar<br>Expirar sesiones   sin trabajar 2:30 minutos |
| /docs          | Ninguno especificado | Tomcat Documentation            | true | 0 | Arrancar   Parar   Recargar   Replegar<br>Expirar sesiones   sin trabajar 2:30 minutos |
| /examples      | Ninguno especificado | Servlet and JSP Examples        | true | 0 | Arrancar   Parar   Recargar   Replegar<br>Expirar sesiones   sin trabajar 2:30 minutos |
| /host-manager  | Ninguno especificado | Tomcat Host Manager Application | true | 0 | Arrancar   Parar   Recargar   Replegar<br>Expirar sesiones   sin trabajar 2:30 minutos |
| /manager       | Ninguno especificado | Tomcat Manager Application      | true | 1 | Arrancar   Parar   Recargar   Replegar<br>Expirar sesiones   sin trabajar 2:30 minutos |
| /shopchain     | Ninguno especificado | Servlet 3+ / JSP Starter        | true | 0 | Arrancar   Parar   Recargar   Replegar<br>Expirar sesiones   sin trabajar 2:30 minutos |

**Desplegar**  
Seleccionar directorio o archivo WAR localizado en servidor

## Despliegue: Despliegue de proyecto en Apache Tomcat



ShopChain  
Gestión de inventarios y logística

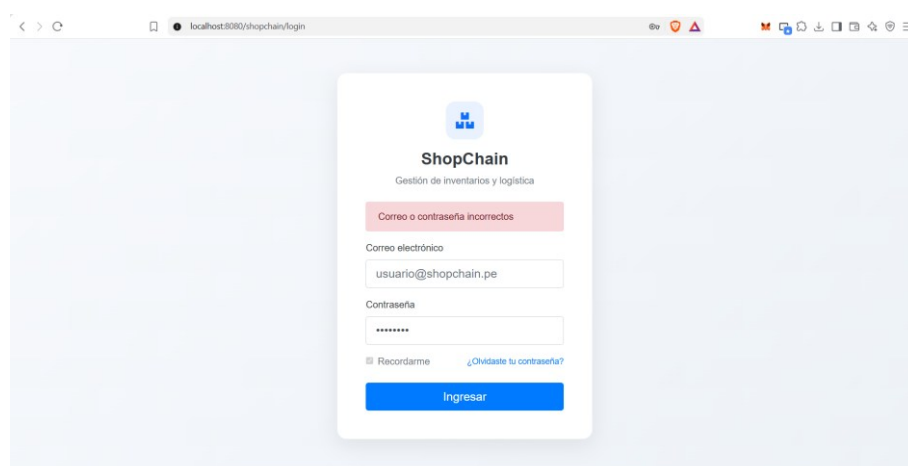
Correo electrónico  
usuario@shopchain.pe

Contraseña  
\*\*\*\*\*

☐ Recordarme    ¿Olvidaste tu contraseña?

Ingresar

## Inicio de sesión: usuario, contraseña



ShopChain  
Gestión de inventarios y logística

Correo o contraseña incorrectos

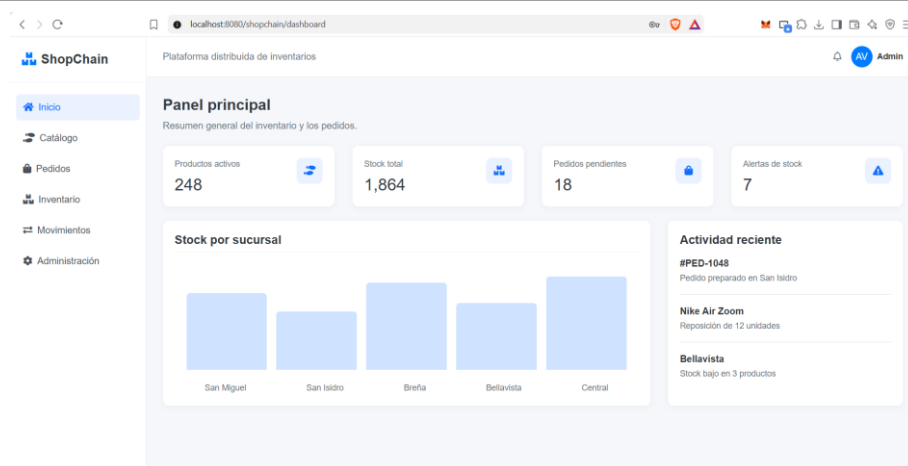
Correo electrónico  
usuario@shopchain.pe

Contraseña  
\*\*\*\*\*

☐ Recordarme    ¿Olvidaste tu contraseña?

Ingresar

## Inicio de sesión: validación de contraseña



**Inicio:** resumen general de productos, pedidos y estado del inventario.

---

## ***CAPITULO 03***

### **Historias de usuario**

#### **HU-01: Inicio de sesión**

**Como** usuario del sistema,  
**quiero** iniciar sesión utilizando mis credenciales,  
**para** acceder únicamente a las funciones correspondientes a mi rol.

**Actor:** Administrador, personal de tienda y personal de almacén.

#### **HU-02: Consultar catálogo de productos**

**Como** usuario del sistema,  
**quiero** consultar el catálogo de productos,  
**para** visualizar los productos disponibles y conocer su información principal.

La aplicación ya contempla un módulo específico de catálogo, detalle de producto y disponibilidad por sucursal.

#### **HU-03: Consultar stock por sucursal**

**Como** personal de tienda,  
**quiero** consultar el stock disponible de un producto en cada sucursal,  
**para** conocer su disponibilidad antes de registrar o atender un pedido.

#### **HU-04: Registrar pedidos**

**Como** personal de tienda,  
**quiero** registrar los productos solicitados por un cliente y seleccionar una sucursal de retiro,  
**para** generar un nuevo pedido dentro de la plataforma.

#### **HU-05: Realizar seguimiento de pedidos**

**Como** usuario del sistema,  
**quiero** consultar el estado de un pedido,  
**para** conocer si se encuentra pendiente, en preparación, listo para retiro, completado o cancelado.

---

#### **HU-06: Registrar movimientos de inventario**

**Como** personal de almacén,  
**quiero** registrar entradas, salidas y reposiciones de productos,  
**para** mantener actualizado el inventario de las sucursales.

Esta funcionalidad ya fue identificada en el diseño inicial como parte del módulo de inventario.

#### **HU-07: Gestionar reposiciones**

**Como** personal de almacén,  
**quiero** identificar productos con stock bajo y registrar reposiciones,  
**para** mantener niveles adecuados de disponibilidad.

#### **HU-08: Gestionar productos**

**Como** administrador,  
**quiero** registrar, modificar y administrar productos,  
**para** mantener actualizado el catálogo de la plataforma.

#### **HU-09: Gestionar sucursales**

**Como** administrador,  
**quiero** registrar y actualizar la información de las sucursales,  
**para** mantener correctamente configurados los puntos donde se controla inventario y se retiran pedidos.

#### **HU-10: Gestionar usuarios**

**Como** administrador,  
**quiero** registrar y administrar usuarios y roles,  
**para** controlar el acceso a las funciones del sistema.

### Criterios de aceptación

| Historia | Criterios de aceptación                                                                                                                             |
|----------|-----------------------------------------------------------------------------------------------------------------------------------------------------|
| HU-01    | El sistema debe validar las credenciales ingresadas. Si son correctas debe permitir el acceso; si son incorrectas debe mostrar un mensaje de error. |
| HU-02    | El sistema debe mostrar el listado de productos y permitir realizar búsquedas y filtros.                                                            |
| HU-03    | Al seleccionar un producto, el sistema debe mostrar el stock correspondiente a cada sucursal.                                                       |
| HU-04    | El sistema debe permitir seleccionar cliente, productos, cantidades y sucursal de retiro antes de registrar un pedido.                              |
| HU-05    | El sistema debe mostrar la información del pedido y su estado actual.                                                                               |
| HU-06    | El sistema debe registrar producto, tipo de movimiento, cantidad, sucursal y observación.                                                           |
| HU-07    | El sistema debe identificar productos con niveles bajos de stock y permitir registrar su reposición.                                                |
| HU-08    | El administrador debe poder registrar, consultar y actualizar los datos de los productos.                                                           |
| HU-09    | El administrador debe poder registrar y actualizar las sucursales habilitadas.                                                                      |
| HU-10    | El administrador debe poder registrar usuarios, asignar roles y modificar su estado.                                                                |

### Requerimientos funcionales

#### RF-01. Autenticación de usuarios

El sistema deberá permitir el inicio de sesión mediante usuario o correo electrónico y contraseña.

#### RF-02. Control de acceso

El sistema deberá restringir las funcionalidades disponibles según el rol asignado al usuario.

#### RF-03. Consulta de productos

El sistema deberá permitir consultar y buscar productos registrados en el catálogo.

#### RF-04. Detalle de producto

El sistema deberá mostrar nombre, categoría, precio, descripción y disponibilidad del producto.

#### RF-05. Consulta de inventario

El sistema deberá permitir consultar el stock de los productos por sucursal.

#### RF-06. Registro de pedidos

El sistema deberá permitir crear pedidos asociando clientes, productos, cantidades y sucursal de retiro.

---

**RF-07. Seguimiento de pedidos**

El sistema deberá permitir consultar el estado actual de los pedidos.

**RF-08. Registro de movimientos de stock**

El sistema deberá permitir registrar entradas, salidas y reposiciones.

**RF-09. Actualización de inventario**

Cada movimiento de stock deberá reflejarse en la disponibilidad del producto correspondiente.

**RF-10. Gestión de productos**

El sistema deberá permitir registrar y actualizar productos.

**RF-11. Gestión de sucursales**

El sistema deberá permitir registrar y administrar las sucursales.

**RF-12. Gestión de usuarios**

El sistema deberá permitir registrar usuarios, asignar roles y gestionar su estado.

**RF-13. Alertas de stock**

El sistema deberá permitir identificar productos con stock bajo o sin stock.

**Requerimientos no funcionales**

**RNF-01. Rendimiento**

Las operaciones comunes de consulta deberán responder en un tiempo adecuado para una aplicación web de uso administrativo.

**RNF-02. Disponibilidad**

La plataforma deberá encontrarse disponible durante los horarios de operación de las sucursales y del almacén central.

**RNF-03. Seguridad**

Las funcionalidades internas deberán requerir autenticación previa.

**RNF-04. Integridad**

Las operaciones relacionadas con inventario no deberán producir cantidades de stock inconsistentes.

**RNF-05. Compatibilidad**

La aplicación deberá funcionar correctamente en navegadores web modernos.

**RNF-06. Diseño adaptable**

La interfaz deberá poder utilizarse adecuadamente en diferentes resoluciones de pantalla.

**RNF-07. Mantenibilidad**

La solución deberá organizarse por responsabilidades y módulos para facilitar futuras modificaciones.

### RNF-08. Trazabilidad

Los movimientos importantes de inventario deberán conservar información suficiente para identificar el producto, sucursal, tipo de movimiento y responsable.

## Restricciones técnicas

Las principales restricciones son:

- El frontend deberá desarrollarse utilizando **Angular**.
- El backend deberá implementarse utilizando **Java**, dentro de las alternativas consideradas en el curso.
- La comunicación entre frontend y backend deberá realizarse mediante **servicios REST**.
- La solución deberá utilizar una base de datos relacional.
- La aplicación Angular será independiente de las vistas JSP.
- La lógica de negocio no deberá implementarse directamente en los componentes Angular.
- El acceso a los datos deberá realizarse desde el backend.
- El sistema deberá poder ejecutarse en los ambientes definidos para el proyecto académico.
- El código deberá mantenerse dentro del repositorio del proyecto.

## Restricciones económicas

- Se utilizarán herramientas y tecnologías que puedan emplearse sin costos adicionales para el desarrollo académico.
- Se priorizarán frameworks y librerías de código abierto.
- Durante la etapa inicial se utilizarán ambientes locales para desarrollo y pruebas.
- No se considera inicialmente la contratación de infraestructura de alta disponibilidad.
- El diseño permitirá que posteriormente la solución pueda migrarse a infraestructura con mayores capacidades.

## Restricciones legales y de seguridad

La plataforma manejará información de usuarios, clientes, productos, sucursales y operaciones internas, por lo que deberán considerarse medidas destinadas a proteger dicha información.

Se establecen las siguientes restricciones:

- Solo usuarios autenticados podrán acceder a las funciones internas.
- Cada usuario deberá acceder únicamente a las funciones autorizadas para su rol.
- Se deberán validar los datos enviados desde el frontend antes de procesarlos.
- La aplicación deberá evitar exponer información sensible mediante mensajes de error.
- La información personal almacenada deberá tratarse de acuerdo con la normativa de protección de datos personales aplicable.
- Las comunicaciones entre cliente y servidor deberán poder utilizar HTTPS en un ambiente productivo.

## Usabilidad, interoperabilidad y mantenibilidad

**Usabilidad:** La interfaz deberá mantener una navegación sencilla y consistente, esto incluye:

- menú lateral permanente;
- nombres claros para cada módulo;
- formularios con campos identificados;
- mensajes de validación visibles;
- estados representados mediante etiquetas;
- diseño consistente entre pantallas.

**Interoperabilidad:** La comunicación entre Angular y Java deberá realizarse mediante una API REST utilizando estructuras de datos estándar, esto permitirá que el frontend permanezca desacoplado del backend y que, en el futuro, otros clientes puedan consumir los mismos servicios.

**Mantenibilidad:** La solución deberá organizarse en módulos y componentes independientes, en Angular ya se ha definido una separación, ya contemplada “**Estructura de componentes frontend**”. En el backend se buscará una separación equivalente entre responsabilidades de presentación, negocio y persistencia.

## Tiempo, escalabilidad y sostenibilidad

**Tiempo:** El desarrollo se realizará de manera incremental de acuerdo con las semanas establecidas en el curso. Cada etapa deberá producir un incremento funcional que permita continuar con el desarrollo de los siguientes módulos.

**Escalabilidad:** Aunque la primera versión contempla únicamente las sucursales definidas para el caso de estudio, el sistema deberá permitir incorporar posteriormente:

- nuevas sucursales
- mayor cantidad de productos
- nuevos usuarios
- mayor número de pedidos
- nuevos módulos

**Sostenibilidad:** La solución deberá utilizar tecnologías mantenibles y ampliamente utilizadas, evitando dependencias innecesarias que dificulten su evolución.

## CAPITULO 04

### Descripción de la arquitectura propuesta

La solución estará compuesta por tres capas principales:

1. Frontend: aplicación desarrollada con Angular.
2. Backend: aplicación desarrollada con Java utilizando Spring Boot.
3. Persistencia: base de datos relacional accedida mediante JPA/Hibernate.

La comunicación entre Angular y el backend Java se realizará mediante una API REST, intercambiando información principalmente en formato JSON.

Esta separación permitirá que Angular se encargue exclusivamente de la presentación e interacción con el usuario, mientras que Java será responsable de las reglas de negocio, validaciones y acceso a los datos.



El frontend mantiene la estructura modular definida anteriormente mediante los módulos **auth**, **home**, **catalog**, **orders**, **inventory** y **administration**, además de los elementos reutilizables agrupados en **core** y **shared**.

## Comparación de alternativas para el backend

### Servlets

Los Servlets permiten procesar directamente las solicitudes HTTP recibidas por el servidor. Su funcionamiento generalmente requiere definir rutas, recibir parámetros, procesar las solicitudes y generar las respuestas desde clases que extienden HttpServlet.

| Ventajas                                                    | Desventajas                                                                                          |
|-------------------------------------------------------------|------------------------------------------------------------------------------------------------------|
| Permiten comprender directamente el funcionamiento de HTTP. | Requieren mayor cantidad de configuración manual.                                                    |
| Tienen pocas abstracciones.                                 | El crecimiento del proyecto puede producir una mayor cantidad de código repetitivo.                  |
| Son adecuados para aplicaciones pequeñas.                   | La implementación de APIs REST requiere más trabajo manual.                                          |
| Permiten trabajar directamente con Apache Tomcat.           | La gestión de dependencias entre componentes debe realizarse manualmente.                            |
|                                                             | Conforme aumenta el sistema, resulta más difícil mantener una separación clara de responsabilidades. |

Los Servlets fueron utilizados durante las primeras etapas del proyecto para validar conceptos como autenticación y despliegue en Apache Tomcat. Sin embargo, no representan la arquitectura definitiva del frontend ni del backend.

### Spring MVC / Spring Boot

Spring permite organizar la aplicación mediante controladores, servicios, repositorios y otros componentes claramente separados. Spring Boot simplifica adicionalmente la configuración necesaria para ejecutar una aplicación Spring y facilita especialmente la creación de APIs REST.

| Ventajas                                              | Desventajas                                                    |
|-------------------------------------------------------|----------------------------------------------------------------|
| Facilita la construcción de servicios REST.           | Presenta una curva de aprendizaje mayor que Servlets.          |
| Reduce configuración repetitiva.                      | Introduce más abstracciones.                                   |
| Permite organizar el sistema en capas.                | Requiere comprender los principales componentes del framework. |
| Facilita la inyección de dependencias.                |                                                                |
| Se integra directamente con JPA/Hibernate.            |                                                                |
| Permite validar las solicitudes recibidas.            |                                                                |
| Facilita el mantenimiento y crecimiento del proyecto. |                                                                |

## Comparación

| Criterio                  | Servlets                             | Spring Boot                                   |
|---------------------------|--------------------------------------|-----------------------------------------------|
| Manejo HTTP               | Manual                               | Simplificado mediante controladores           |
| API REST                  | Requiere mayor implementación manual | Soporte directo                               |
| Inyección de dependencias | Manual                               | Integrada                                     |
| Organización por capas    | Debe implementarse manualmente       | Facilita su implementación                    |
| Validaciones              | Manuales                             | Integración con mecanismos de validación      |
| Persistencia              | JDBC principalmente                  | JDBC y JPA/Hibernate                          |
| Escalabilidad del código  | Adecuada para aplicaciones pequeñas  | Adecuada para aplicaciones con varios módulos |
| Integración con Angular   | Posible                              | Más directa mediante API REST                 |

## Alternativa seleccionada

Para la versión definitiva se seleccionará **Spring Boot** como tecnología principal del backend.

## Elección de la estrategia de persistencia

### JDBC

JDBC permite ejecutar directamente sentencias SQL desde Java. Por ejemplo, las operaciones de consulta, registro y actualización deben implementarse manualmente utilizando conexiones, sentencias SQL y procesamiento de resultados.

| Ventajas                                                                       | Desventajas                                                                    |
|--------------------------------------------------------------------------------|--------------------------------------------------------------------------------|
| Mayor control sobre las consultas SQL.                                         | Produce mayor cantidad de código repetitivo.                                   |
| Menor nivel de abstracción.                                                    | Requiere mapear manualmente los resultados.                                    |
| Facilita comprender directamente la interacción entre Java y la base de datos. | Requiere gestionar consultas SQL.                                              |
|                                                                                | Puede incrementar la complejidad cuando existen muchas entidades relacionadas. |

## JPA/Hibernate

JPA permite representar las tablas de la base de datos mediante clases Java denominadas entidades. Hibernate puede utilizarse como implementación de JPA para realizar el mapeo entre objetos Java y registros de la base de datos.

| Ventajas                                                          | Desventajas                                                                                       |
|-------------------------------------------------------------------|---------------------------------------------------------------------------------------------------|
| Reduce código repetitivo.                                         | Introduce un nivel adicional de abstracción.                                                      |
| Facilita las operaciones CRUD.                                    | Requiere comprender el funcionamiento del mapeo objeto-relacional.                                |
| Permite definir relaciones entre entidades.                       | Algunas consultas complejas pueden requerir JPQL, consultas nativas u optimizaciones adicionales. |
| Se integra directamente con Spring.                               |                                                                                                   |
| Facilita la organización del código mediante repositorios.        |                                                                                                   |
| Reduce la necesidad de escribir SQL para las operaciones comunes. |                                                                                                   |

## Comparación

| Criterio                   | JDBC                      | JPA/Hibernate                      |
|----------------------------|---------------------------|------------------------------------|
| Consultas SQL              | Principalmente manuales   | Generadas para operaciones comunes |
| Mapeo de resultados        | Manual                    | Automático                         |
| CRUD                       | Implementación manual     | Simplificado                       |
| Relaciones entre entidades | Manejo manual             | Mediante anotaciones               |
| Código repetitivo          | Mayor                     | Menor                              |
| Integración con Spring     | Disponible                | Directa mediante Spring Data JPA   |
| Mantenibilidad             | Menor en sistemas grandes | Mayor                              |

## Estrategia seleccionada

Se utilizará **JPA/Hibernate** para implementar la capa de persistencia.

## Diseño de la comunicación frontend-backend

| Angular                                    | Spring Boot                               |
|--------------------------------------------|-------------------------------------------|
| Presentar las interfaces.                  | Recibir solicitudes.                      |
| Capturar información.                      | Validar datos.                            |
| Realizar validaciones básicas de interfaz. | Autenticar usuarios.                      |
| Manejar la navegación.                     | Ejecutar reglas de negocio.               |
| Enviar solicitudes al backend.             | Consultar o modificar información.        |
| Mostrar las respuestas obtenidas.          | Acceder a la base de datos.               |
|                                            | Devolver las respuestas correspondientes. |

La comunicación utilizará: **HTTP + REST + JSON**



La respuesta podría tener una estructura similar a:

```
{
  "id": 1,
  "name": "Nike Air Max",
  "category": "Running",
  "price": 499.90
}
```

## Justificación técnica de las decisiones tomadas

La combinación de Angular + Spring Boot + JPA/Hibernate fue seleccionada porque permite establecer responsabilidades claramente diferenciadas.

### Angular

Angular será utilizado porque permite construir una aplicación frontend basada en componentes. La organización modular del proyecto facilita mantener separadas las funcionalidades de autenticación, catálogo, pedidos, inventario y administración.

### Spring Boot

Spring Boot permitirá desarrollar el backend mediante una arquitectura organizada en responsabilidades:



## JPA/Hibernate

JPA permitirá representar los elementos principales del dominio mediante entidades Java y simplificará las operaciones de persistencia.

## API REST

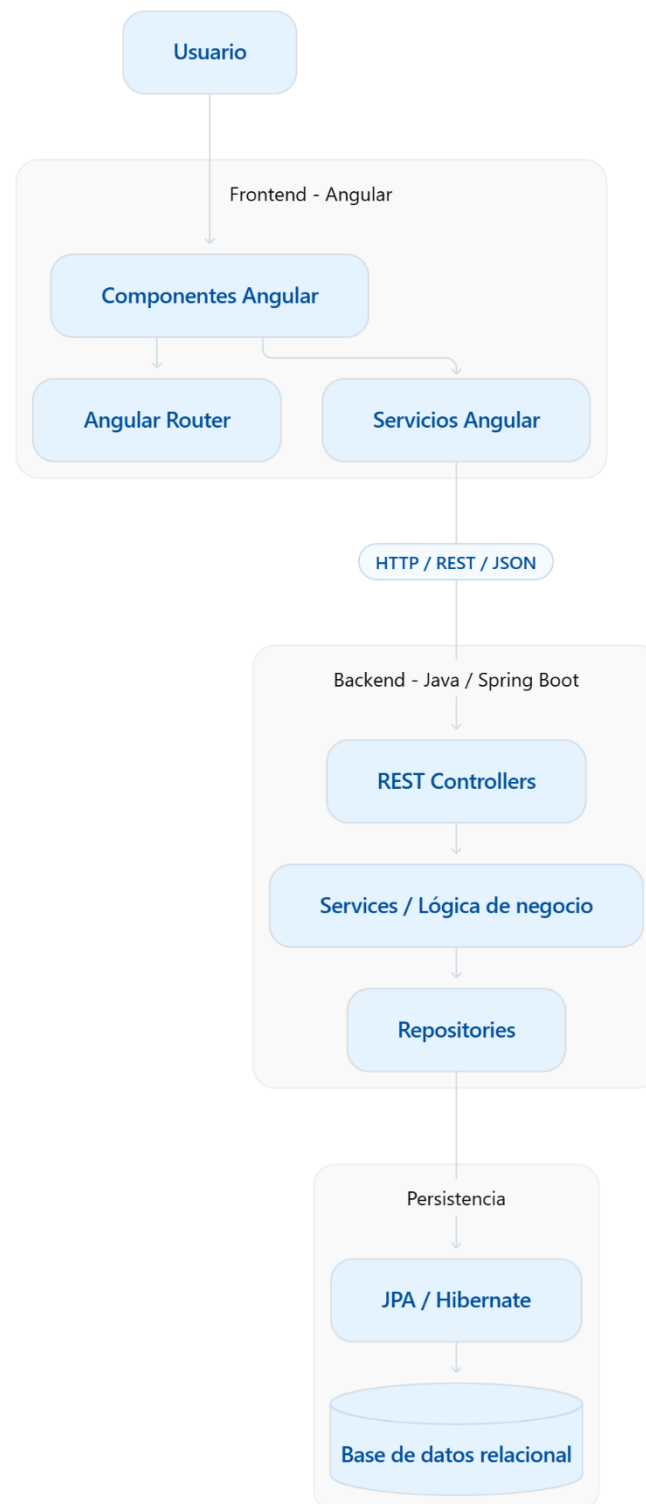
La utilización de una API REST permitirá mantener Angular desacoplado de Java.

## Herramientas y librerías para utilizar

Las principales herramientas consideradas para el desarrollo son:

| Herramienta / Tecnología | Función                                       |
|--------------------------|-----------------------------------------------|
| Angular                  | Desarrollo del frontend                       |
| TypeScript               | Lenguaje utilizado por Angular                |
| Angular Router           | Navegación entre pantallas                    |
| Angular HttpClient       | Comunicación con la API REST                  |
| Java                     | Lenguaje del backend                          |
| Spring Boot              | Desarrollo del backend                        |
| Spring Web               | Implementación de servicios REST              |
| Spring Data JPA          | Acceso a datos                                |
| Hibernate                | Implementación ORM                            |
| Maven                    | Gestión de dependencias Java                  |
| Base de datos relacional | Persistencia del sistema                      |
| Apache Tomcat            | Servidor utilizado durante el desarrollo Java |
| Git                      | Control de versiones                          |
| GitHub                   | Repositorio del proyecto                      |
| Visual Studio Code       | Desarrollo y revisión del código              |
| Postman o Insomnia       | Pruebas de la API REST                        |

## Diagrama de arquitectura del sistema



---

## **Análisis de viabilidad técnica dentro de las restricciones del curso**

La arquitectura propuesta es técnicamente viable porque utiliza tecnologías empresariales y no requiere incorporar plataformas externas para construir las funcionalidades principales.

1. Angular permite implementar las interfaces definidas en el diseño del sistema.
2. Spring Boot permite construir los servicios necesarios
3. JPA/Hibernate permite implementar la persistencia utilizando una base de datos relacional.

Además, la arquitectura puede desarrollarse progresivamente.

Inicialmente Angular puede consumir datos de prueba, como ocurre actualmente en el prototipo, y posteriormente dichos servicios pueden ser reemplazados por llamadas HTTP reales sin modificar completamente los componentes.

La organización propuesta también permite incrementar posteriormente la cantidad de productos, sucursales y usuarios, aspecto considerado desde la justificación inicial del proyecto.

Por lo tanto, la solución:

**Resulta adecuada para implementar el alcance planteado y mantiene una separación clara entre presentación, lógica de negocio y persistencia.**